VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA", BELAGAVI - 590 018

DATA SCIENCE AND BIG DATA ANALYTICS (21AD71)

LABORATORY REPORT

Submitted by

Vishesh Hadimani **4SF21AD060**

In partial fulfilment of the requirements for the VII semester

of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

at

SAHYADRI

COLLEGE OF ENGINEERING & MANAGEMENT

An Autonomous Institution

Adyar, Mangaluru - 575 007

Academic Year: 2024 - 25

SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

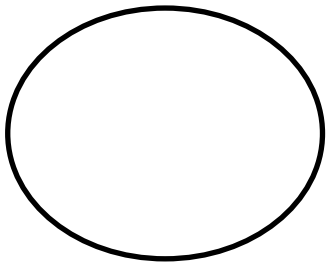
CERTIFICATE

This is to certify that Mr. Vishesh Hadimani bearing USN: 4SF21AD060 has satisfactorily completed the course of experiments in Data Science and Big Data Analytics Laboratory (21AD71) in partial fulfilment of the requirements of VII semester of Bachelor of Engineering Degree Course in Artificial Intelligence & Data Science as prescribed by the Visvesvaraya Technological University during the year 2024-25.

Date:

Internal Assessment Marks

Faculty In charge



Experiment Details

Experiment No.	Experiment Name	Date of Execution	Page No.	Marks (10)
1	Line Chart for Student Data	13-09-2024	01	
2	Histogram for Frequency distribution in mtcars.csv	19-09-2024	02	
3	Data Cleaning for given Dataset	26-09-2024	03	
4	K-Means clustering using MapReduce	03-10-2024	05	
5	Count Frequency of given word using MapReduce	10-10-2024	07	
6	Max, min, mean of Iris Dataset using MapReduce	15-11-2024	08	
7	K-Means and Hierarchical Clustering using Spark	22-11-2024	11	
8	Maximum temperature using Spark in Java	29-11-2024	13	

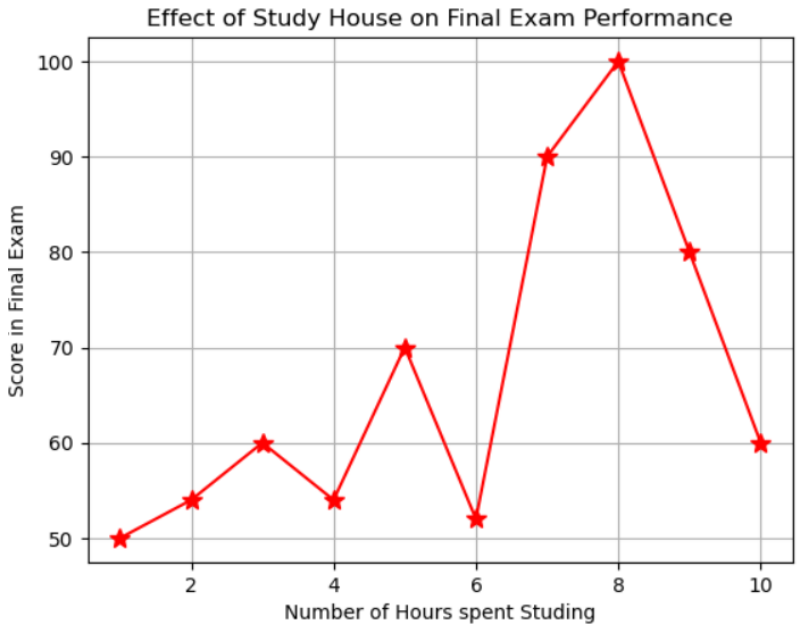
Lab Experiment - 1

1. **A study was conducted to understand the effect of number of hours the students spend studying on their performance in the final exam. Write a code to plot line chart with number of hours studying on X Axis and score in final exam on the Y Axis. Use a red star as the point character; label the axis and give the plot a title.**

Program:

```
import matplotlib.pyplot as plt
hours = [1,2,3,4,5,6,7,8,9,10]
scores = [50,54,60,54,70,52,90,100,80,60]
plt.plot(hours,scores,marker = '*', color = 'red', linestyle = '-', markersize =10)
plt.xlabel('Number of Hours spent Studing')
plt.ylabel('Score in Final Exam')
plt.title('Effect of Study House on Final Exam Performance')
plt.grid(True)
plt.show()
```

Output:



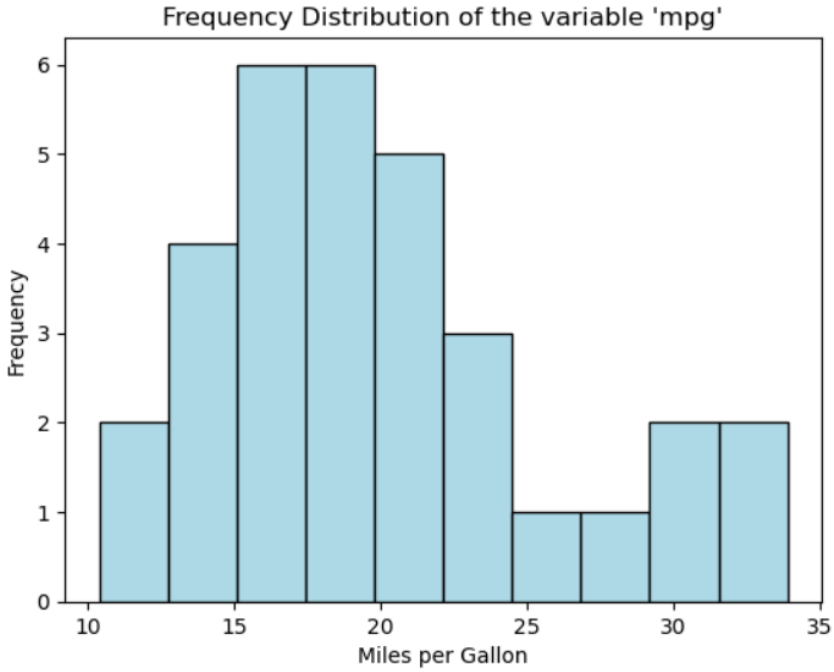
Lab Experiment - 2

2. For the given dataset 'mtcars.csv' plot a histogram to check the frequency distribution of the variable 'mpg'.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv('mtcars.csv')

plt.hist(data['mpg'], color = 'lightblue', edgecolor = 'black')
plt.xlabel('Miles per Gallon')
plt.ylabel('Frequency')
plt.title("Frequency Distribution of the variable 'mpg'")
plt.show()
```

Output:



Lab Experiment - 3

3. Consider the books dataset BL-Flickr-Images-Book.csv from Kaggle which contains information about books. Write a program to demonstrate the following: Program:

1. **Importing the data into a DataFrame**
2. **Find and drop the columns which are irrelevant for the book information**
3. **Change the index of the DataFrame**
4. **Tidy up the fields in the data such as date of publication with the help of simple regular experession**
5. **Combine str methods with NumPy to clean columns**

Program:

```
import pandas as pd

import numpy as np


# Importing the data into a DataFrame

df = pd.read_csv("BL-Flickr-Images-Book.csv")


# Find and drop the columns which are irrelevant for the book information

icols = ['Edition Statement','Contributors','Corporate Author','Corporate Contributors','Former owner','Engraver','Flickr URL','Shelfmarks','Issuance type']

df.drop(columns = icols, inplace = True)


# Change the index of the DataFrame

df.set_index('Identifier', inplace=True)


# Tidy up the fields in the data such as date of publication with the help of simple regular experession

df['Date of Publication'] = df['Date of Publication'].str.extract(r'(\d{4})')
```

Combine str methods with NumPy to clean columns

df['Place of Publication'] = np.where(df['Place of Publication'].str.contains('London'), 'London', df['Place of Publication'])

df['Place of Publication'] = np.where(df['Place of Publication'].str.contains('Oxford'), 'Oxford', df['Place of Publication'])

print("\nFinal Cleaned DataFrame:")

print(df.head())

Output:

```
Final Cleaned DataFrame:
           Place of Publication Date of Publication              Publisher  \
Identifier
206                      London               1879       S. Tinsley & Co.
216                      London               1868           Virtue & Co.
218                      London               1869  Bradbury, Evans & Co.
472                      London               1851          James Darling
480                      London               1857   Wertheim & Macintosh

                                                    Title     Author
Identifier
206                           Walter Forbes. [A novel.] By A. A      A. A.
216         All for Greed. [A novel. The dedication signed...  A., A. A.
218         Love the Avenger. By the author of “All for Gr...  A., A. A.
472         Welsh Sketches, chiefly ecclesiastical, to the...  A., E. S.
480         [The World in which I live, and my place in it...  A., E. S.
```

Lab Experiment - 4

4. Implement K-means clustering using MapReduce.

Program:

```
import numpy as np

def initialize_centroids(data, k):
    indices = np.random.choice(len(data), size=k, replace=False)
    return data[indices]

def assign_clusters(data, centroids):
    clusters = [[] for _ in range(len(centroids))]
    for point in data:
        distances = np.linalg.norm(point - centroids, axis=1)
        closest_centroid = np.argmin(distances)
        clusters[closest_centroid].append(point)
    return clusters

def update_centroids(clusters):
    new_centroids = []
    for cluster in clusters:
        if cluster:
            new_centroids.append(np.mean(cluster, axis=0))
    return np.array(new_centroids)

def kmeans(data, k, max_iters=100):
    centroids = initialize_centroids(data, k)
    for _ in range(max_iters):
```

```
        clusters = assign_clusters(data, centroids)  # Map step
        new_centroids = update_centroids(clusters)  # Reduce step

        if np.allclose(centroids, new_centroids, atol=1e-6):
            break
        centroids = new_centroids
    return centroids, clusters

if __name__ == "__main__":
    np.random.seed(42)
    data = np.random.rand(100, 2)
    k = 3
    centroids, clusters = kmeans(data, k)

    print("Final centroids:")
    print(centroids)
    print("Cluster sizes:")
    for i, cluster in enumerate(clusters):
        print(f"Cluster {i}: {len(cluster)} points")
```

Output:

```
Final centroids:
[[0.8039633  0.57026999]
 [0.18520943 0.72228065]
 [0.36376248 0.20008043]]
Cluster sizes:
Cluster 0: 38 points
Cluster 1: 28 points
Cluster 2: 34 points
```

Lab Experiment - 5

5. Give a MapReduce Program to calculate the frequency of a given word in a given file.

Program:

```
with open('lorem.txt', 'r') as f:
    text = f.read()

target_word = "lorem"

def mapper(line):
    words = line.strip().split()
    return [(word, 1) for word in words if word == target_word]

def reducer(counts):
    total = 0
    for count in counts:
        total += count
    return total

mapped = [pair for line in text.split('\n') for pair in mapper(line)]

counts = [count for word, count in mapped]

frequency = reducer(counts)

print(f"The word '{target_word}' appears {frequency} time(s) in the given text.")
```

Output:

```
The word 'lorem' appears 10 time(s) in the given text.
```

Lab Experiment - 6

6. Develop a Map Reduce program to calculate the maximum, minimum, and mean values of sepal length, sepal width, petal length, and petal width from the iris flower dataset. Visualize the relationship between sepal length and sepal width and also between petal length and petal width.

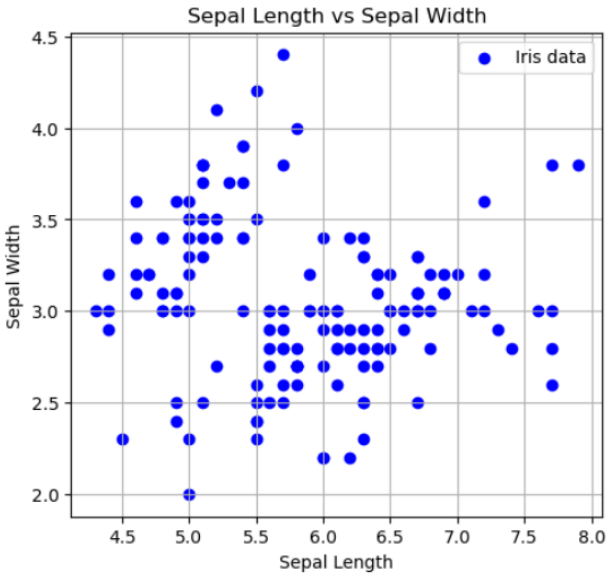
Program:

```
import numpy as np
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

iris = load_iris()
X = iris.data

def map_function(data):
    return {
        'sepal_length': data[0],
        'sepal_width': data[1],
        'petal_length': data[2],
        'petal_width': data[3]
    }

def reduced_function(results):
    stats = {}
    for feature in ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']:
        values = [result[feature] for result in results]
        stats[feature] = {
            'min': np.min(values),
            'max': np.max(values),
            'mean': np.mean(values)
```

```
        }
    return stats

mapped_results = [map_function(data) for data in X]

min_max_mean_values = reduced_function(mapped_results)
for feature, values in min_max_mean_values.items():
    print(f"{feature}: Min= {values['min']}, Max= {values['max']}, Mean= {values['mean']}")

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:, 1], c='blue', label='Iris data')
plt.grid(True)
plt.title("Sepal Length vs Sepal Width")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.legend()

plt.subplot(1,2,2)
plt.scatter(X[:,2], X[:, 3], c='red', label='Iris data')
plt.grid(True)
plt.title("Petal Length vs Petal Width")
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.legend()

plt.show()
```

Output:

```
sepal_length: Min= 4.3, Max= 7.9, Mean= 5.843333333333334
sepal_width: Min= 2.0, Max= 4.4, Mean= 3.0573333333333337
petal_length: Min= 1.0, Max= 6.9, Mean= 3.7580000000000005
petal_width: Min= 0.1, Max= 2.5, Mean= 1.1993333333333336
```



Lab Experiment - 7

7. Implement K-Means and Hierarchical Clustering technique using Spark.

Program:

```
from pyspark.sql import SparkSession
from pyspark.ml.clustering import KMeans,BisectingKMeans
from pyspark.ml.evaluation import ClusteringEvaluator

spark = SparkSession.builder.appName("KMeansExample").getOrCreate()
dataset = spark.read.format("libsvm").load("kmeans_data.txt")

kmeans = KMeans().setK(2).setSeed(1)
model = kmeans.fit(dataset)

predictions = model.transform(dataset)
predictions.show()

evaluator = ClusteringEvaluator()
silhouette = evaluator.evaluate(predictions)
print(f"Silhouette with squared euclidean distance = {silhouette}")

centers = model.clusterCenters()
print("Cluster Centers: ")
for center in centers:
    print(center)

bkm = BisectingKMeans().setK(2).setSeed(1)
model = bkm.fit(dataset)

predictions = model.transform(dataset)
```

```
predictions.show()

evaluator = ClusteringEvaluator()
silhouette = evaluator.evaluate(predictions)
print(f"Silhouette with squared euclidean distance = {silhouette}")

centers = model.clusterCenters()
print("Cluster Centers: ")
for center in centers:
    print(center)
```

Output:

```
+-----+--------------------+----------+
|label|            features|prediction|
+-----+--------------------+----------+
|  0.0|(3,[0,1,2],[0.1,0...|         0|
|  1.0|(3,[0,1,2],[0.4,0...|         0|
|  0.0|(3,[0,1,2],[0.7,0...|         1|
|  1.0|(3,[0,1,2],[1.0,1...|         1|
+-----+--------------------+----------+

Silhouette with squared euclidean distance = 0.7230769230769221
Cluster Centers:
[0.25 0.35 0.45]
[0.85 0.95 1.05]
+-----+--------------------+----------+
|label|            features|prediction|
+-----+--------------------+----------+
|  0.0|(3,[0,1,2],[0.1,0...|         0|
|  1.0|(3,[0,1,2],[0.4,0...|         0|
|  0.0|(3,[0,1,2],[0.7,0...|         1|
|  1.0|(3,[0,1,2],[1.0,1...|         1|
+-----+--------------------+----------+

Silhouette with squared euclidean distance = 0.7230769230769221
Cluster Centers:
[0.25 0.35 0.45]
[0.85 0.95 1.05]
```

Lab Experiment - 8

8. Develop a Java application to find the maximum temperature using Spark.

Program:

```
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.SparkConf;
import org.apache.log4j.Level;
import org.apache.log4j.Logger;

public class MaxTemperature {
   public static void main(String[] args) {
      Logger.getLogger("org.apache.spark").setLevel(Level.WARN);
      SparkConf conf = new SparkConf().setAppName("Max Temperature").setMaster("local");
      JavaSparkContext sc = new JavaSparkContext(conf);

      String inputFile = args[0];
      JavaRDD<String> lines = sc.textFile(inputFile);

      JavaRDD<Double> temperatures = lines.map(new Function<String, Double>() {
         @Override
         public Double call(String line) {
            String[] parts = line.split(",");
            return Double.parseDouble(parts[1]);
         }
      });

      Double maxTemperature = temperatures.reduce((a, b) -> Math.max(a, b));
```

```
        System.out.println("Maximum Temperature: " + maxTemperature);

        sc.stop();
    }
}
```

Output:

```
$ spark-submit --class MaxTemperature --master local target/max-temperature-1.0-
SNAPSHOT.jar temperature_data.txt
Maximum Temperature: 34.8
```